

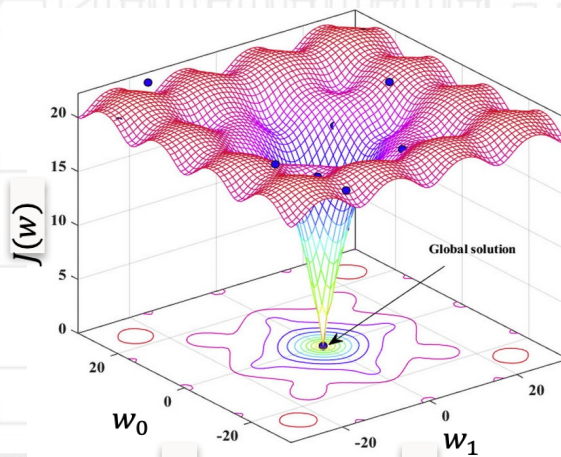


Module 1

Regularization

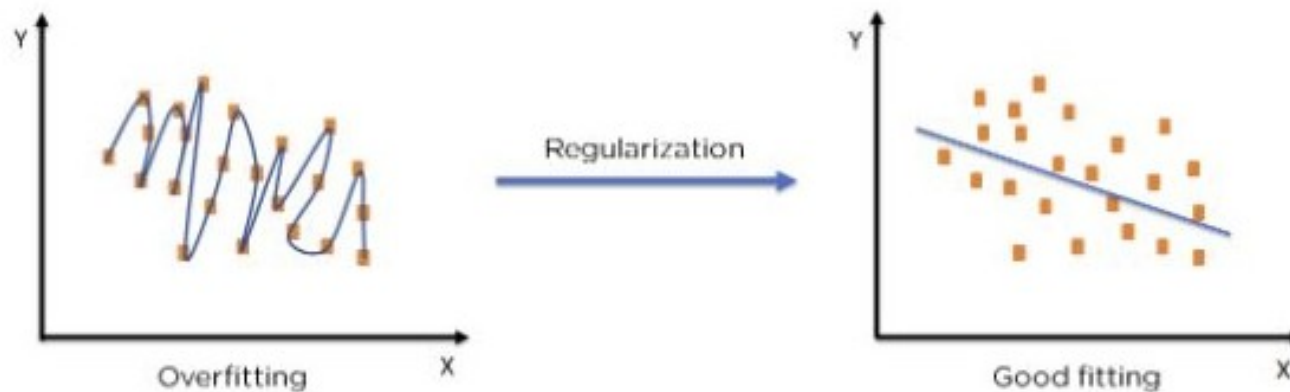


Regularization



Regularization

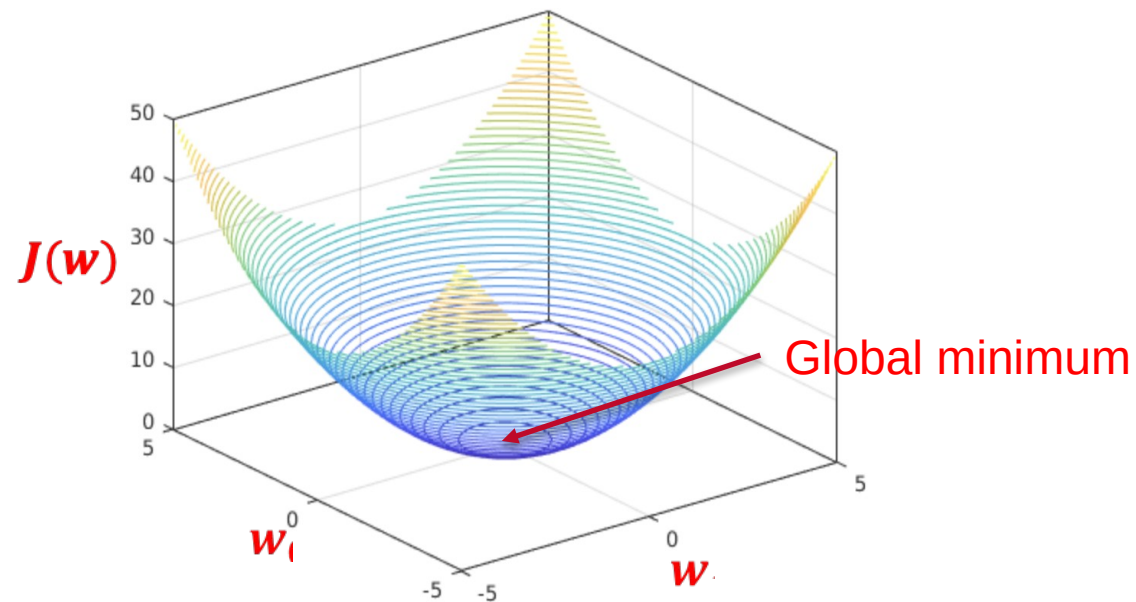
- ❑ Regularization is a set of techniques used in ML to improve a model's ability to generalize to new data.
 - ❑ Cost Function Penalties (linear models + gradient boosting)
 - ❑ Pruning (trees)
 - ❑ Dropout + Cost Function Penalties (neural networks)



Linear Regression Cost Function

- E.g., for linear regression, the cost function is convex:

$$J(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$



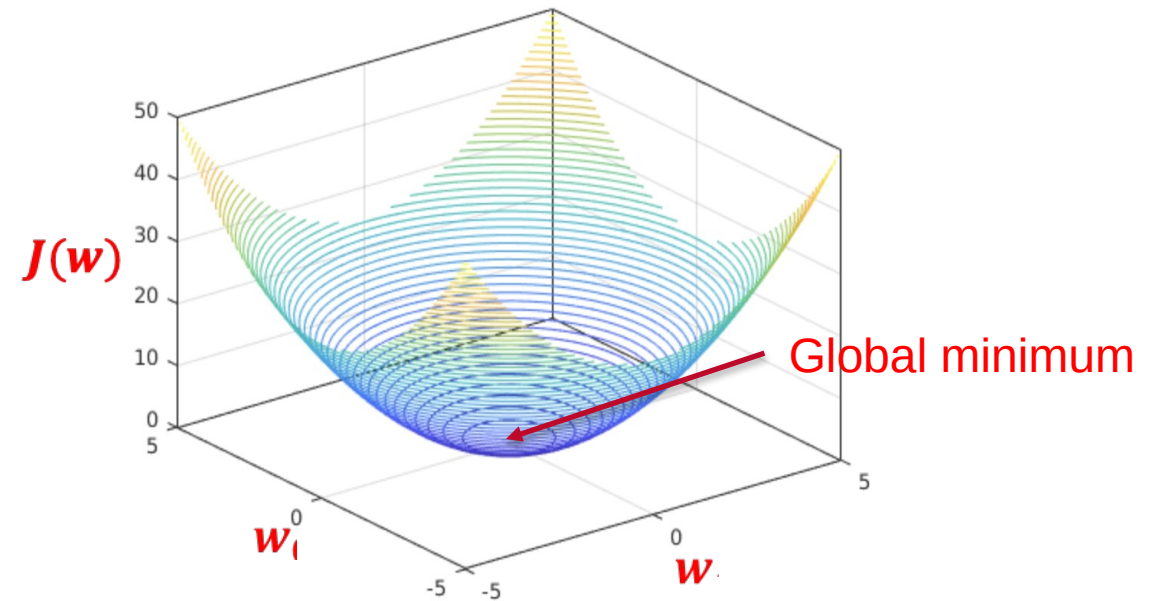
Linear Regression Cost Function

□ E.g., for linear regression, the cost function is convex:

$$J(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

$$\mathbf{X} = \begin{bmatrix} 1 & X_{11} & X_{12} & \cdots & X_{1k} \\ 1 & X_{21} & X_{22} & \cdots & X_{2k} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 1 & X_{n1} & X_{n2} & \cdots & X_{nk} \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_k \end{bmatrix}$$



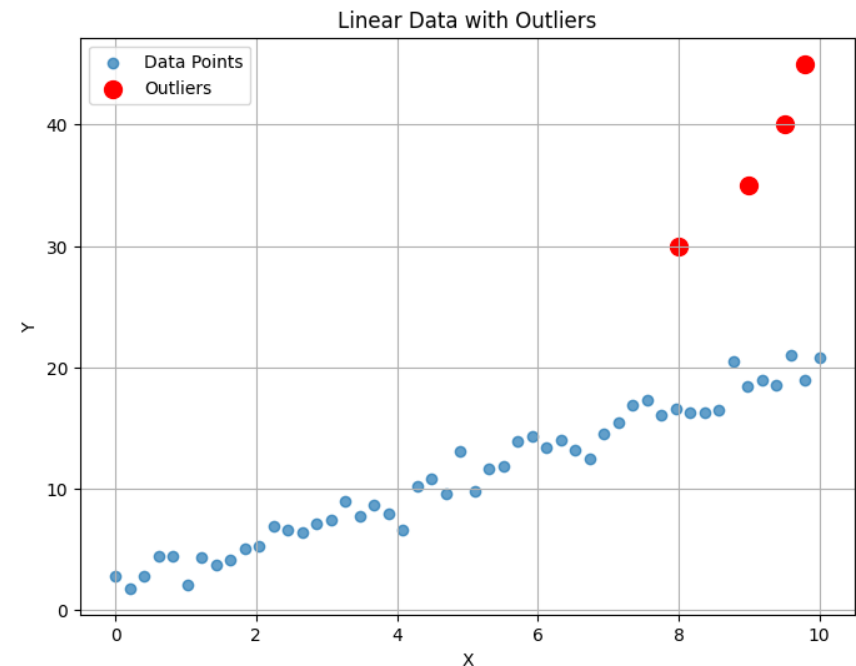
We focus on regularizing (controlling) the weights

Linear Regression Cost Function

□ E.g., for linear regression, the cost function is convex:

$$J(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$



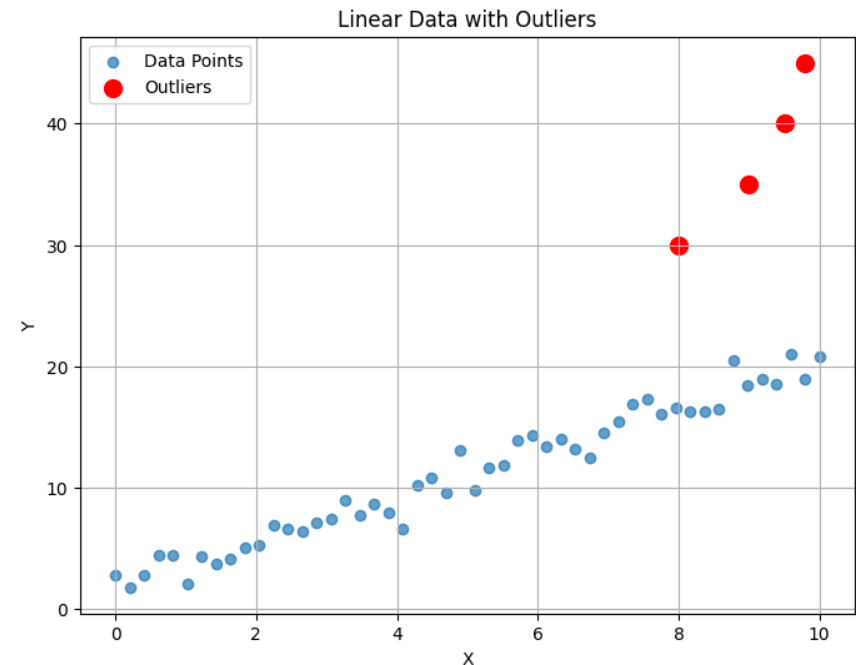
We focus on regularizing (controlling) the weights

Linear Regression Cost Function

□ E.g., for linear regression, the cost function is convex:

$$J(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

$$\mathbf{X} = \begin{bmatrix} 1 & X_{11} & X_{12} & \cdots & X_{1k} \\ 1 & X_{21} & X_{22} & \cdots & X_{2k} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 1 & X_{n1} & X_{n2} & \cdots & X_{nk} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_k \end{bmatrix}$$



We focus on regularizing (controlling) the weights

Linear Regression Cost Function

- ▣ E.g., for linear regression, the objective function is:

$$\min_{\mathbf{w}} J(\mathbf{w})$$

- ▣ Substituting the cost function:

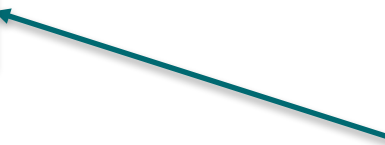
$$\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

Ridge Regression - Adding an L₂ Penalty

- ▣ Ridge Regression uses a modification to the cost function
- ▣ The objective will become

$$\min_w ||\mathbf{y} - \mathbf{X}\mathbf{w}||_2^2 + \alpha ||\mathbf{w}||_2^2$$

L2 penalty



- ▣ Forces the model to have parameters closer to zero
- ▣ α controls how much penalty. Can be any value $\alpha > 0$
- ▣ If $\alpha = 0$, then we are back to regular regression



Python

Ridge Regression with Cross-Validation

- ▣ The objective for Ridge Regression will become

$$\min_w \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_2^2$$

L2 penalty

- ▣ Determining hyper-parameter α is important
- ▣ We can use cross-validation to determine α

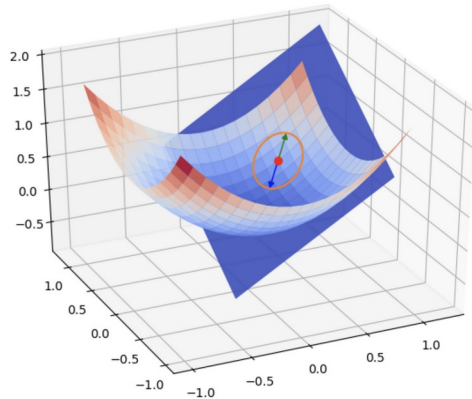


Python

Ridge Regression Gradient

- The gradient for Ridge Regression is:

$$\frac{\delta J(\mathbf{w})}{\delta \mathbf{w}} = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\alpha\mathbf{w}$$



- The cost function is still convex (FAST COMPUTATIONALLY)
- The minimum can be found with the formula below:

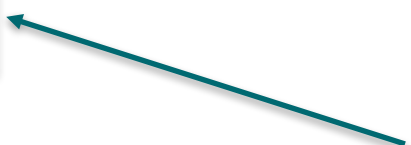
$$\mathbf{w} = (\alpha I + \mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{y})$$

Lasso Regression - Adding an L₁ Penalty

- ▣ Lasso Regression uses a modification to the cost function
- ▣ The objective will become

$$\min_{\mathbf{w}} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_1$$

L1 penalty



- ▣ Forces the model to have parameters equal to zero if not essential
- ▣ α controls how much penalty. Can be any value $\alpha > 0$
- ▣ If $\alpha = 0$, then we are back to regular regression



Python

Lasso Regression with Cross-Validation

- ▣ The objective for Ridge Regression will become

$$\min_{\mathbf{w}} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_1$$

← L1 penalty

- ▣ Determining hyper-parameter α is important
- ▣ We can use cross-validation to determine α



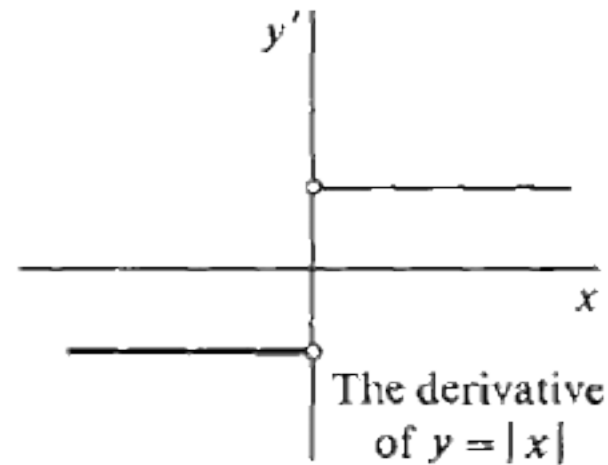
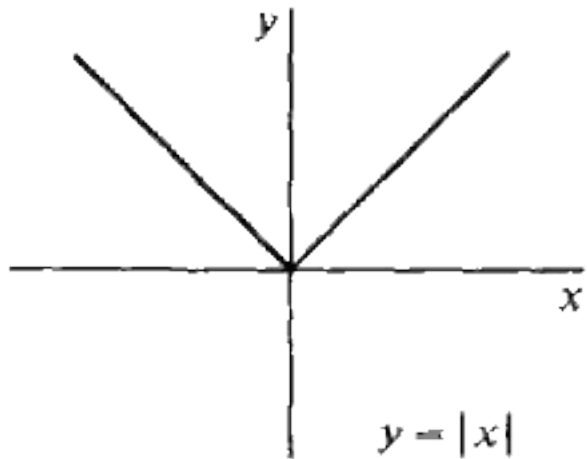
Python

Lasso Regression with Cross-Validation

▣ The objective for Ridge Regression will become

$$\min_w \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_1$$

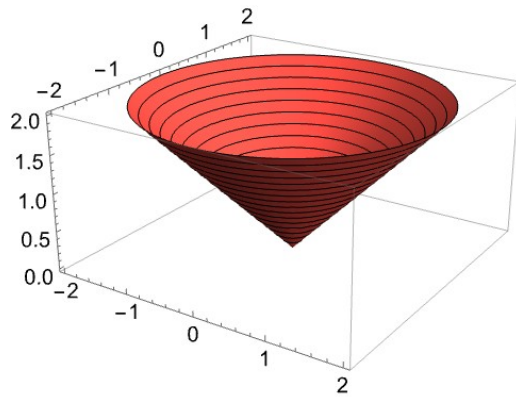
L1 penalty



Lasso Regression Gradient

- ❑ The gradient for Ridge Regression does not exist at zero:

$$\frac{\delta J(\mathbf{w})}{\delta \mathbf{w}} = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + \alpha \text{sign}(w)$$



- ❑ The cost function is not strictly convex
- ❑ We use coordinate descent and optimization to obtain minimum

ElasticNet Regression - L_1 and L_2 Penalty

- ElasticNet Regression uses a modification to the cost function
- The objective will become

$$\min_w \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha L_1 \text{Ratio} \|\mathbf{w}\|_1 + 0.5\alpha (1 - L_1 \text{Ratio}) \|\mathbf{w}\|_2^2$$

L1 and L2 penalties

- Forces the model to have parameters equal to zero (unimportant)
- Forces the model to have parameters closer to zero (higher order)
- α controls how much penalty. Can be any value $\alpha > 0$
- If $\alpha = 0$, then we are back to regular regression



Python

Huber Regression

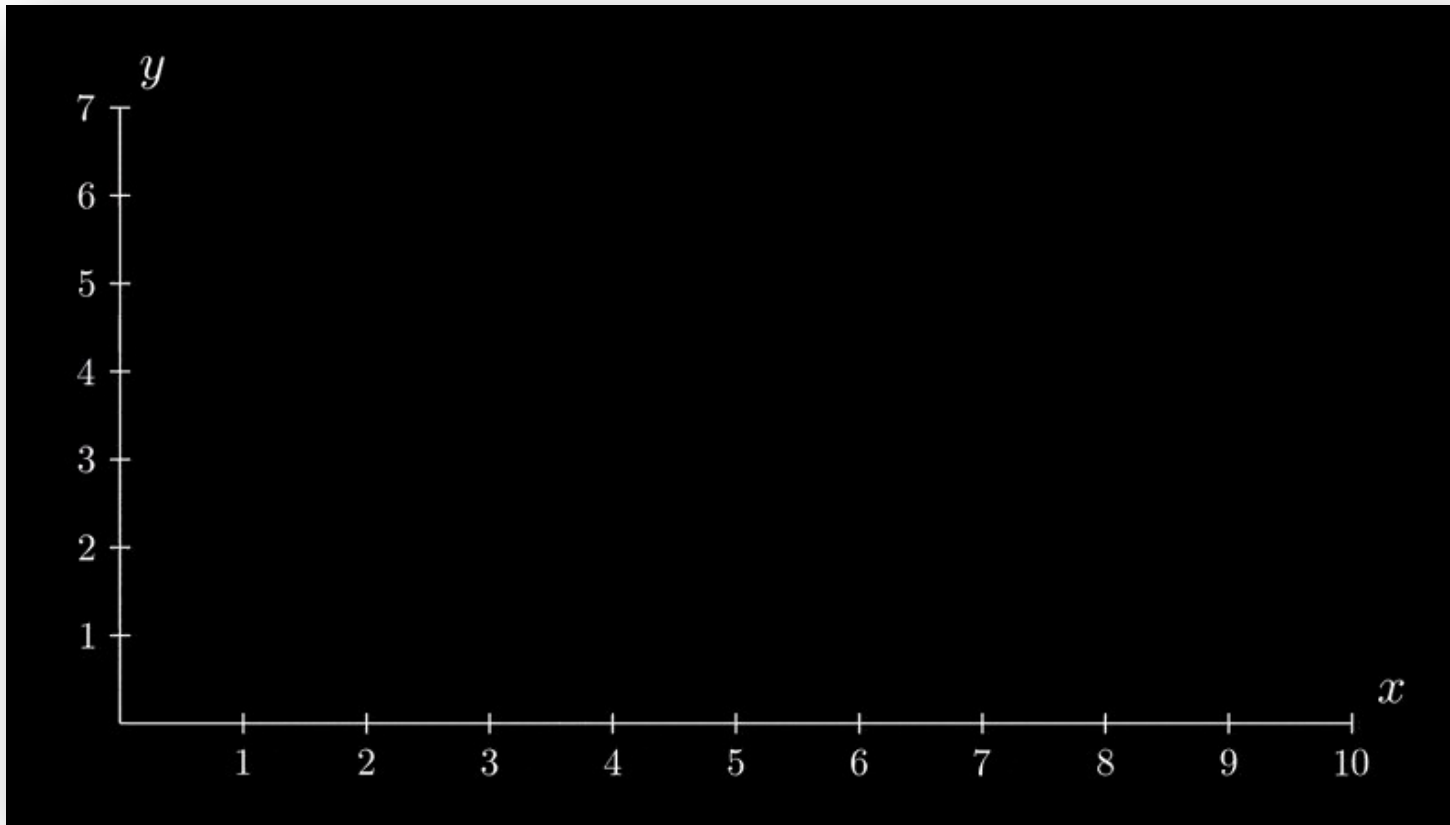
- Huber Regression uses a modification to the cost function
- The objective will become

$$\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 \text{ for observations with } \left| \frac{y - \mathbf{X}\mathbf{w}}{\sigma} \right| < \epsilon$$

$$\min_{\mathbf{w}} |y - \mathbf{X}\mathbf{w}| \text{ for observations with } \left| \frac{y - \mathbf{X}\mathbf{w}}{\sigma} \right| > \epsilon$$

- Meant to be use for datasets with big outliers

Huber Regression





Python